Introduction to Object-oriented Programming (OOP)

HDDS1103

Lecture 01

Intended Learning Outcomes

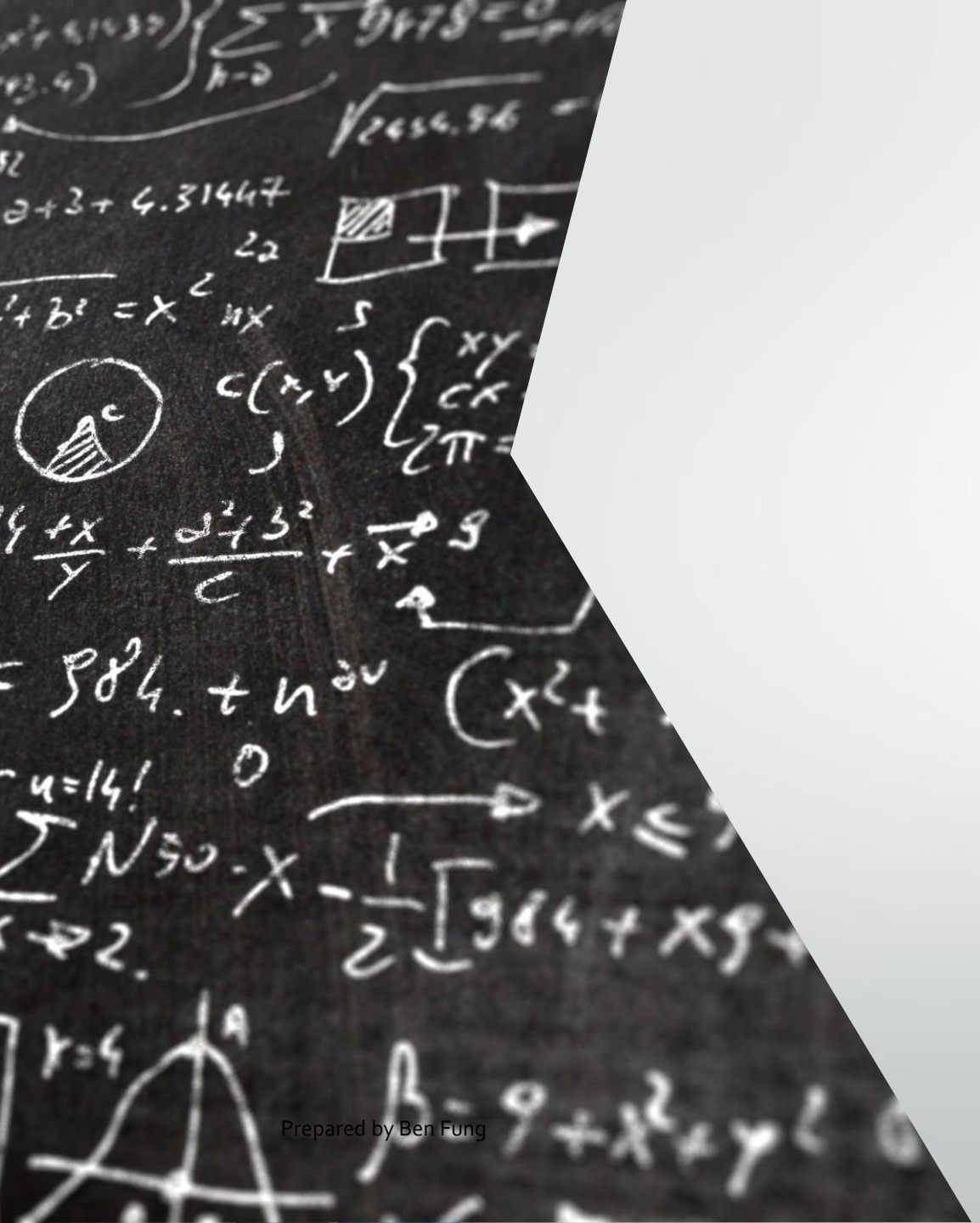
After completing the lesson, students should be able to:

- Describe different programming development environments
- Describe the features of OOP languages
- Use command line & IDE for program compilation and execution
- Write comments and documentation for java programs

Programming Languages

Programming Language

- Programming language
  - System of notation for writing computer programs
- Program
  - Set of instructions a computer follows to perform a task
- Algorithm
  - a step-by-step procedure or a set of rules for solving a specific problem.



Evolution of Programming Languages

First Generation Languages (1GL)

- Written in machine language
  - Using binary numbers
- Each CPU architecture has their own machine language
  - Intel x86 / ARM processor
- Machine friendly, unfriendly to human
- Example: (x = 3 + 4)

```
B0 03 04 04 88 06 B4 0E 04 30 CD 10 B4 4C 30 CD 21
```

Second Generation Language (2GL)

- Assembly language (low-level language) is developed
  - Make it easier to code
  - Still processor dependent

Example (x = 3 + 4) (in x86)

```
assembly
section .data
    x db 0

section .text
    global _start

_start:
    mov al, 3       ; Move the value 3 into register AL
    add al, 4       ; Add the value 4 to the value in register AL

    mov [x], al     ; Move the result to variable 'x'

    ; Display the value of 'x'
    mov ah, 0x0E    ; Move the value 0x0E (BIOS function for printing) into register AH
    add al, 48      ; Convert the value to its ASCII representation
    int 0x10        ; Invoke the BIOS interrupt 0x10 (video services)

    ; Terminate the program
    mov ah, 0x4C    ; Move the value 0x4C (DOS function for program termination) into register AH
    xor al, al      ; Set AL to 0 (return code)
    int 0x21        ; Invoke the DOS interrupt 0x21
```

Third Generation Language (3GL)

- High-level programming language
- Closer to human language
- More abstract than assembly language

- Examples:
  - COBOL (1959), Fortran (1957)
  - C (1972)
  - C++ (1985)
  - Java (1995)
  - Python (1991)

Hello World, again

```cpp
// helloworld.cpp
#include <iostream>
using namespace std;
int main() {
  std::cout << "Hello, world!" << std::endl;
  return 0;
}
```

```python
# Hello World in Python
if __name__ == "__main__":
    print("Hello, world!")
```

```java
// HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Key features of 3GL

- High-level abstraction
- Portability
- Readability
- Modularity
- Abundant libraries and frameworks
- Strong typing and type checking
- Support for structured programming
- Object-oriented programming (OOP) support

Fourth GL, Fifth GL

4GL

- More user-friendly,
- focus on what to do (declarative)
  - Not how to do (procedural)
- Simplified coding - for non-programmer
- Example
  - Structured Query Language (SQL)

5GL

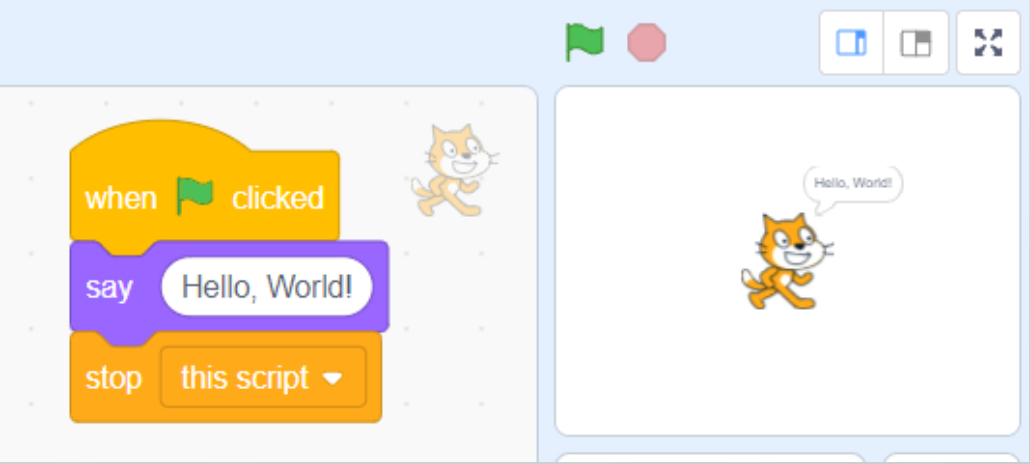
- further abstraction
  - focus on high-level problem-solving and design
- AI integration,
- domain-specific languages,
- code generation
- visual programming
- Example: Scratch

Hello World, again & again

4GL (SQL)

```
/* Hello World in SQL */
SELECT 'Hello, world!';
```

5GL (Scratch)



Scratch 3.0 Download: https://scratch.mit.edu/download

Key Features of Object-oriented Programming

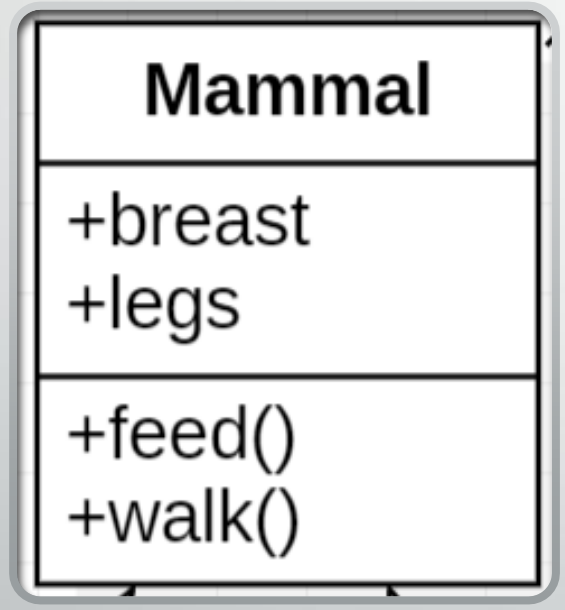
```python
account_balance = 1000
def deposit(amount):
# . . .
def withdraw(amount):
    global account_balance
    if amount <= account_balance:
        account_balance -= amount
        print(f"Withdrew: {amount}. New Balance: {account_balance}")
    else:
        print("Insufficient funds for withdrawal.")
print(f"Initial balance: {account_balance}")

deposit(500)
withdraw(200)

# . . .
account_balance = -9999
# . . .
# The state is now corrupted, and functions may behave unexpectedly.
withdraw(50)
```

Source code: messy_acct_bal.py

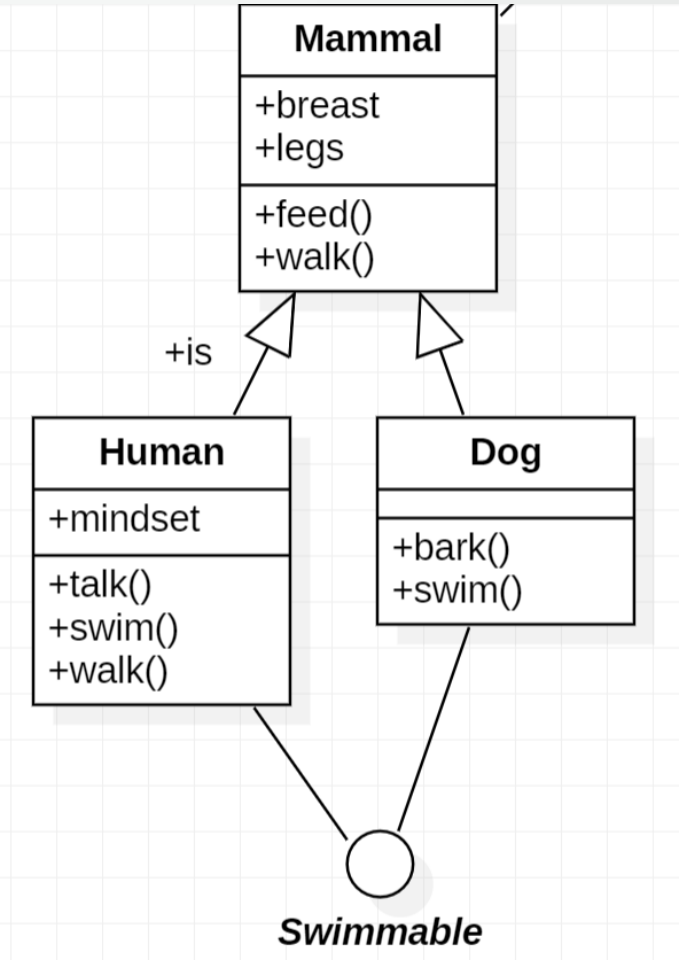
Object-oriented Programming (OOP)



- Programming paradigm organizes code into objects
  - Object: Instance of class
  - Class: Encapsulate data (attribute) and behavior (method)
- Example:
  - Ben (Object) is an example of Mammal (Class).
  - Mammal (class) has legs (attribute) and they can walk (method)

Key Features of OOP

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction



Exercise: BankAccount Class

- Draw a design for a BankAccount.
- The diagram should show:
  - A private variable to hold the account balance.
  - A public method called deposit().
  - A public method called withdraw().

Java

Key Features

Java Introduction

- High-level programming language
- By Sun Microsystems (now by Oracle)
- Key features:
  - Object-oriented
  - Platform Independent
  - Robust - garbage collection
  - Extensive Standard Library

Portability

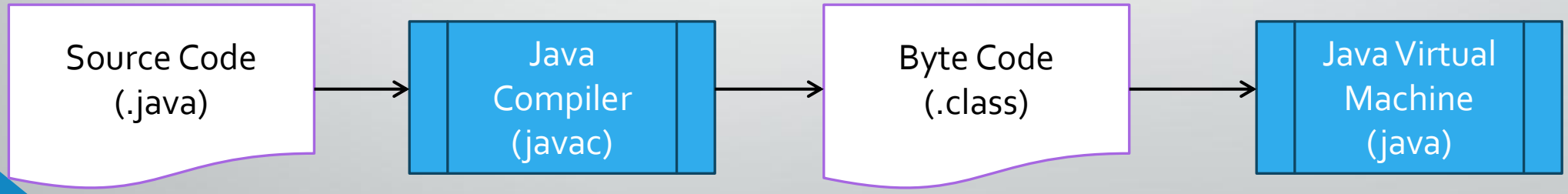
- Source Portability
  - Program written
    - on one platform
  - Run in different platform
    - Little or no modification
- Java Solution
  - “Write One, Run Everywhere”
  - Java Byte code run on JVM
- JVM supports
  - Windows, UNIX, Linux, Mac OS X

Compiled Language vs Interpreted Language

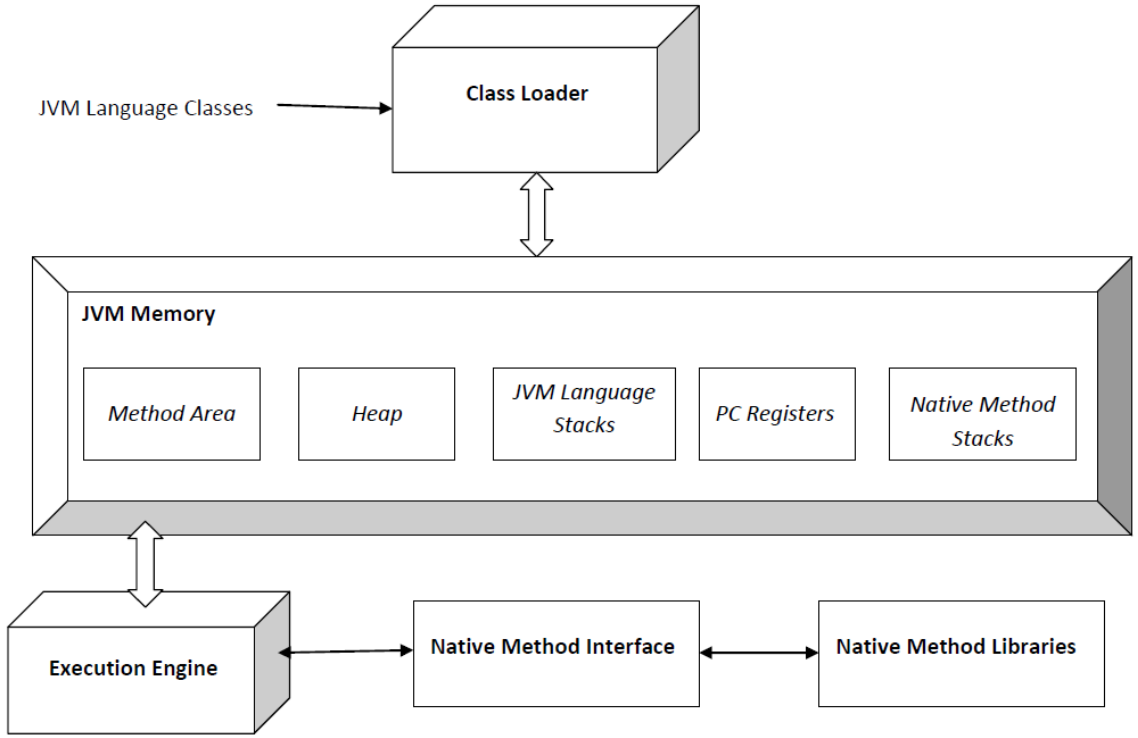
Category	Compiled Language	Interpreted Language
Process	Source code is translated into machine code by a compiler.	Source code is executed line by line by an interpreter.
Execution	The compiled program is executed directly by the computer's processor.	The interpreter reads and executes the code on-the-fly.
Performance	Generally, offers faster execution and better performance.	Generally slower execution due to the overhead of interpreting the code at runtime.
Portability	Programs are platform-specific and need to be recompiled for different platforms.	Programs are more portable as the same source code can be run on different platforms with compatible interpreters.

Compilation and Interpreted Process

- Java Compiler
  - Compile Java Source Code into platform-independent byte code
- Java Virtual Machine (JVM)
  - Read the bytecode, and translate into machine code at runtime
  - Just-in-time (JIT) compilation



JVM architecture (Java SE 7)



- By Michelle Ridomi - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=35963523

Java Runtime Environment (JRE)

- Software layer that runs on top of OS
  - Provides class libraries & other resources
- JRE architecture
  - **ClassLoader**: dynamically loads all classes necessary to run a Java program
  - **Bytecode verifier**: ensures the format & accuracy of Java code
  - **Interpreter**: creates instance of JVM
    - Execute Java program natively on the underlying machine
- Download: https://www.java.com/en/download/

Java Development Kit (JDK)

- Distribution of Java technology by Oracle
  - Implements the Java Language Specification, Java Virtual Machine
  - Standard Edition (Java SE) of the Java Application Programming Interface (API)
  - https://www.oracle.com/java/technologies/downloads/
- Open JDK
  - Open-source builds of JDK
  - https://jdk.java.net/21/

Java platform editions

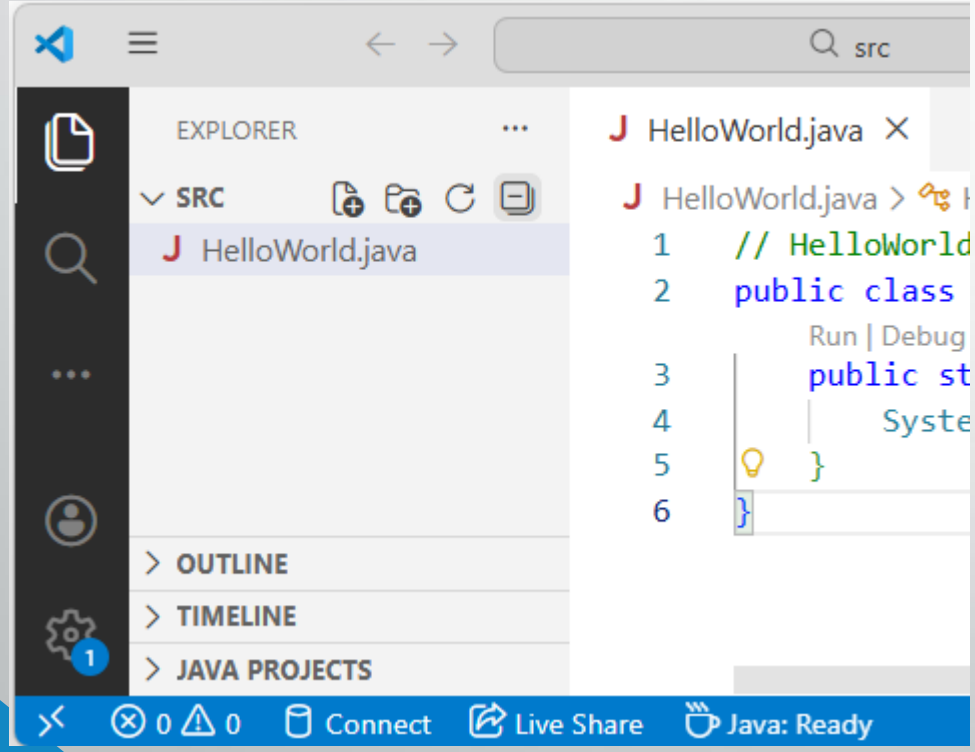
- Other than the Standard Edition (Java SE 21)
- Release for different purposes
  - Java EE : Enterprise Edition (version 10 – Java 11 & 17 support)
  - Java ME: Micro Edition for embedded system
  - Java FX: Deploy rich client application
    - GUI programming

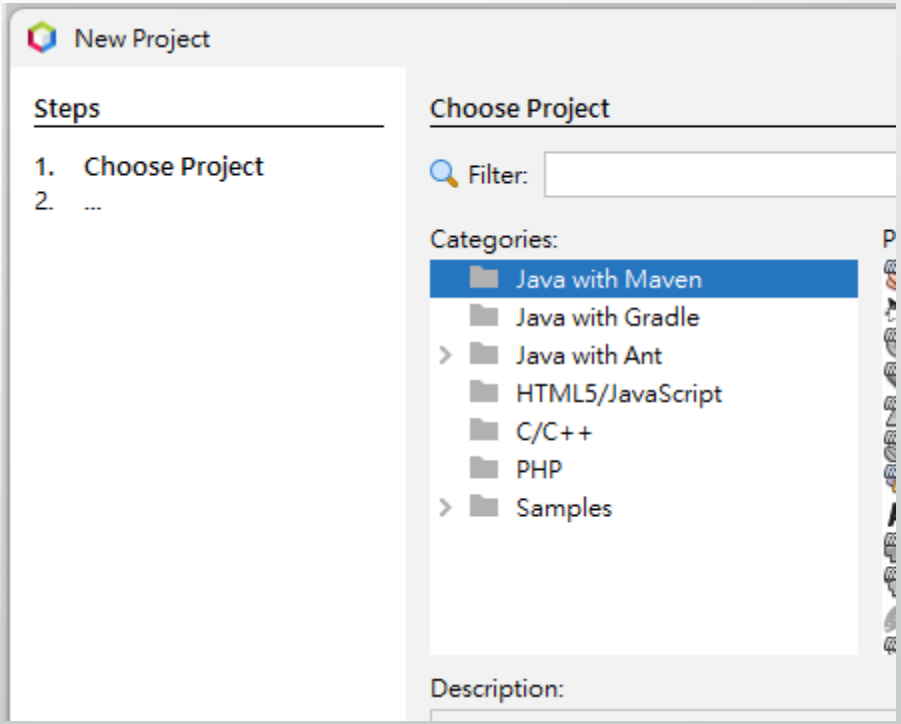
Source: https://www.oracle.com/java/technologies/downloads/archive/

Programming Development Environments for Java

Programming Development Environments

- **Eclipse, IntelliJ IDEA, VS Code, NetBean, Command Line Tools**





Details: https://en.wikipedia.org/wiki/Comparison_of_integrated_development_environments#Java

First Java Program HelloWorld.java

```java
// Comment using //
// Public class with same File name (HelloWorld.java)
public class HelloWorld {
    // defining publicly seen class (static) function (method) called main.
    // Accepting text (String) array ([]) argument (args)
    public static void main(String[] args) {
        // Using System (class), out's (static member) println method
        // to print the String (") "Hello World!")
        System.out.println("Hello, World!");
    }
}
```

Compile using Java Compiler

- javac [options] [sourcefiles-or-classnames]
- javac command
  - reads Java source files
  - compiles them into class files
    - that run on the Java Virtual Machine.
- Result: .class files for each class
- trigger in Command Prompt / Terminal (Mac)
- Example: javac HelloWorld.java

```
命令提示字元

C:\>cd OOP-L1

C:\OOP-L1>dir
 磁碟區 C 中的磁碟是 WIN10
 磁碟區序號:  EE79-7D0A

 C:\OOP-L1 的目錄

2024-01-11  13:52    <DIR>          .
2024-01-11  13:52    <DIR>          ..
2024-01-10  17:36               454 HelloWorld.java
               1 個檔案            454 位元組
               2 個目錄  36,777,005,056 位元組可用

C:\OOP-L1>javac HelloWorld.java

C:\OOP-L1>dir
 磁碟區 C 中的磁碟是 WIN10
 磁碟區序號:  EE79-7D0A

 C:\OOP-L1 的目錄

2024-01-11  13:53    <DIR>          .
2024-01-11  13:53    <DIR>          ..
2024-01-11  13:53               427 HelloWorld.class
2024-01-10  17:36               454 HelloWorld.java
```

More: https://docs.oracle.com/en/java/javase/21/docs/specs/man/javac.html

Execute using JVM

- java – launch a Java application
- java [options] mainclass [args ...]
  - Note: mainclass without file extension
- Example: javac HelloWorld
- Other ways to launch

  java –jar jarfile

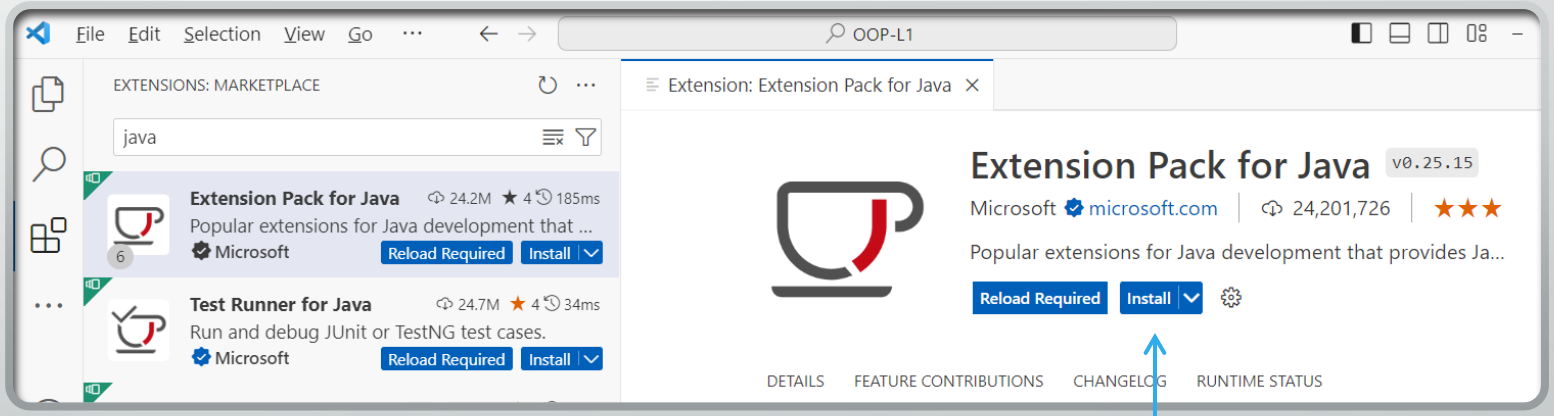
  java –m module

Reference: https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html

Setup VSCode

- Visual Studio Code can execute Java Program
  - JDK need to be installed
  - May install the “Extension Pack For Java” from Microsoft
    - For IntelliSense, Debugging, and Testing

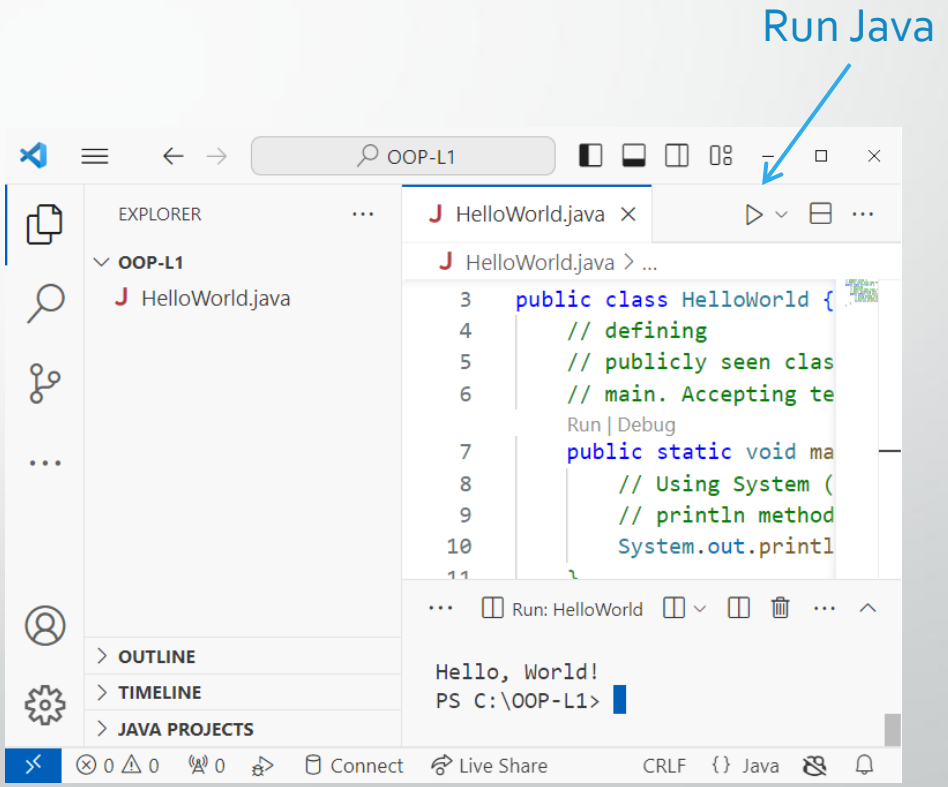
Extension



Install

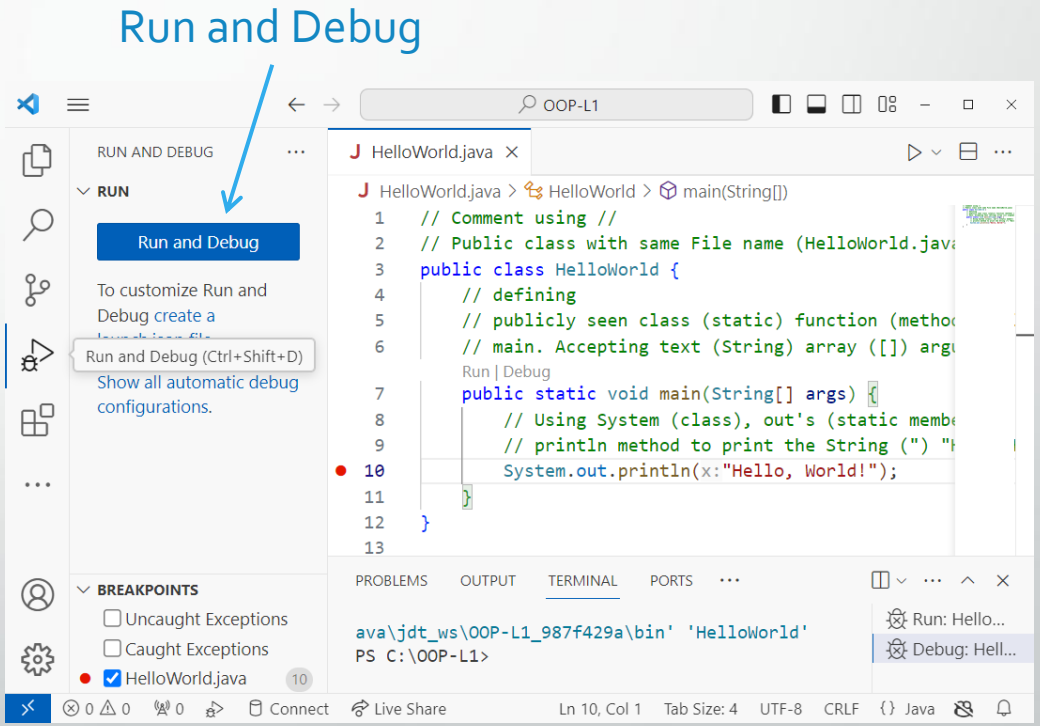
Build & Run Java Program in VS Code

- Open VS Code
- Open Folder containing the .java file
- Open the .java file
- Choose Run > Start without Debugging
  - Shortcut: (Ctrl + F5)
  - Or click Run Java Button



Debugging in VS Code

- With “Extension Pack For Java”
- Debugging like python programs
  - Main menu: Run > Start Debugging (F5)
  - Run and Debug > Run and Debug
- With breakpoints
  - Can step over the program for debugging



Java Program Structure

Revisit HelloWorld.java

```
// Comment using //
// Public class with same File name (HelloWorld.java)
public class HelloWorld {
    // defining publicly seen class (static) function (method) called main.
    // Accepting text (String) array ([]) argument (args)
    public static void main(String[] args) {
        // Using System (class), out's (static member) println method
        // to print the String (") "Hello World!")
        System.out.println("Hello, World!");
    }
}
```

Revisit (HelloWorld.java)

```
// A single line comment, ignored by JVM
```

```
// Public class …
public class HelloWorld {
 …
}
```

Class body

publicly visible class

Class name **must** match with File name for public class

Revisit (HelloWorld.java)

```
public class HelloWorld {


    public static void main(String[] args) {

      …
    }
}
```

Method body

Method header of main method.
Application start point

System.out.println("Hello, World!");

- System.out.println(…);
  - System: System class from java language
  - out: 'standard' output stream
  - println(): print a data to the screen, and a line separator string.
- "Hello, World!": The data to be printed.

Parts of a Java Program

- Comments
  - The line is ignored by the compiler.
  - The comment in the example is a single-line comment. //
- Class Header
  - The class header tells the compiler things about the class such as what other classes can use it (public) and that it is a Java class (class), and the name of that class (HelloWorld).
- Curly Braces
  - When associated with the class header, they define the scope of the class.
  - When associated with a method, they define the scope of the method.

Comments in Java

- To improve code readability, ignored by the compiler.
- Types:
  - Single-line comments
  - Multi-line comments
  - Javadoc comments: for generating API documentation.
- Use comments to explain why
  - not what it does
- Avoid excessive commenting; well-written code often needs few comments.

```
// Single line comment.


/* This is a
   multi-line comment
 */


/**
 * This is a Javadoc comment
 */
```

Parts of a Java Program

- `main` method
  - must be exactly as shown in the example
    - (except the *args* variable name can be programmer defined).
  - line of code that the *java* command will run first.
  - Every ***application*** must have a `main` method.
- Java Statements
  - statements within the `main` method will be executed.

Java Statements

- Statements are separated by semi-colon (;)
- Other elements that do not need semi colons:
  - Comments (ignored)
  - class headers
  - method headers
  - curly braces

Exercise – Self Introduction

Requirements

- Write a program to introduce yourself
  - Full name
  - English Name
  - Target semester GPA (sGPA)

Sample output

```
Full name: FUNG Cheuk Luk
English name: Ben
Target sGPA: 3.1415926
```

Short Review

- Java is a case-sensitive language.
- All Java programs must be stored with a .java file extension.
- Comments are ignored by the compiler.
- A .java file may contain many classes but may only have one public class.
  - The public class must have the **same name** as the file.

Short Review

- Java applications must have a `main` method.
- For every opening brace (left brace) {
  - a corresponding closing brace (right brace) }
- Statements are terminated with semicolons.

Special Characters

//	double slash	Marks the beginning of a single line comment.
( )	open and close parenthesis	Used in a method header to mark the *parameter list*.
{ }	open and close curly braces	Encloses a group of statements, such as the contents of a class or a method.
" "	quotation marks	Encloses a string of characters, such as a message that is to be printed on the screen
;	semi-colon	Marks the end of a complete programming statement

Java Documentation

- Official documentation
  - all the language and standard library reference
  - https://docs.oracle.com/en/java/javase/21/
- Tutorial
  - From "Get Started" to Server-side programming
  - https://docs.oracle.com/javase/tutorial/tutorialLearningPaths.html